



Python Programming Fundamentals

Dunder Methods and Operator Overloading

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. Dunder (Magic) Methods
2. Overriding `__str__`
3. Overriding `__eq__`
4. Overriding `__lt__`
5. `__repr__`
6. Operator Overloading: `__add__`
7. Summary



Outline

1. Dunder (Magic) Methods

2. Overriding `__str__`

3. Overriding `__eq__`

4. Overriding `__lt__`

5. `__repr__`

6. Operator Overloading: `__add__`

7. Summary



1.1 Python's special method protocol

Methods with double underscores (`__name__`) are called **dunder** or **magic** methods. Python calls them automatically in specific situations.

Method	Trigger	Purpose
<code>__str__</code>	<code>print(obj)</code> , <code>str(obj)</code>	Human-readable string
<code>__repr__</code>	REPL, debugging	Developer string
<code>__eq__</code>	<code>obj1 == obj2</code>	Equality check
<code>__lt__</code>	<code>obj1 < obj2</code>	Less-than comparison
<code>__len__</code>	<code>len(obj)</code>	Length
<code>__add__</code>	<code>obj1 + obj2</code>	Addition



Outline

1. Dunder (Magic) Methods

2. Overriding `__str__`

3. Overriding `__eq__`

4. Overriding `__lt__`

5. `__repr__`

6. Operator Overloading: `__add__`

7. Summary



2. Overriding `__str__`

```
class Product:
    # Without __str__:
    # print(p) → <product.Product object at 0x7f...>

    def __str__(self) -> str:
        return (f"Product [{self.__product_id}]: "
                f"{self.__name} – €{self.__price:.2f}")

p = Product("Apple", 1.99, 101)
print(p)                # Product [101]: Apple – €1.99
print(f"Item: {p}")     # Item: Product [101]: Apple – €1.99
```



Outline

1. Dunder (Magic) Methods
2. Overriding `__str__`
3. Overriding `__eq__`
4. Overriding `__lt__`
5. `__repr__`
6. Operator Overloading: `__add__`
7. Summary



3. Overriding __eq__

```
class Product:
    def __eq__(self, other) -> bool:
        if not isinstance(other, Product):
            return NotImplemented
        return self.__product_id == other.__product_id

p1 = Product("Apple", 1.99, 101)
p2 = Product("Apple", 2.50, 101)    # same ID, different price
p3 = Product("Bread", 2.50, 102)

print(p1 == p2)    # True   (same ID)
print(p1 == p3)    # False  (different IDs)
```

Without __eq__, Python compares by **object identity** (is) which is rarely what you want.



Outline

1. Dunder (Magic) Methods
2. Overriding `__str__`
3. Overriding `__eq__`
4. Overriding `__lt__`
5. `__repr__`
6. Operator Overloading: `__add__`
7. Summary



4.1 Enabling sorting

```
class Product:
    def __lt__(self, other) -> bool:
        if not isinstance(other, Product):
            return NotImplemented
        return self.__price < other.__price

products = [
    Product("Butter", 3.00, 104),
    Product("Apple", 1.99, 101),
    Product("Milk", 1.20, 103),
]

products.sort()      # uses __lt__ – sorts by price
for p in products:
    print(p)
# Product [103]: Milk – €1.20
```



4.1 Enabling sorting

```
# Product [101]: Apple – €1.99  
# Product [104]: Butter – €3.00
```



Outline

1. Dunder (Magic) Methods
2. Overriding `__str__`
3. Overriding `__eq__`
4. Overriding `__lt__`
5. `__repr__`
6. Operator Overloading: `__add__`
7. Summary



5.1 Developer-facing representation

```
class Product:
    def __repr__(self) -> str:
        return (f"Product(name={self.__name!r}, "
                f"price={self.__price!r}, "
                f"product_id={self.__product_id!r})")

p = Product("Apple", 1.99, 101)
print(repr(p))
# Product(name='Apple', price=1.99, product_id=101)

# In a list, Python uses __repr__:
print([p])
# [Product(name='Apple', price=1.99, product_id=101)]
```



Outline

1. Dunder (Magic) Methods

2. Overriding `__str__`

3. Overriding `__eq__`

4. Overriding `__lt__`

5. `__repr__`

6. Operator Overloading: `__add__`

7. Summary



6. Operator Overloading: __add__

```
class Vector:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __add__(self, other: "Vector") -> "Vector":
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self) -> str:
        return f"Vector({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2          # calls v1.__add__(v2)
print(v3)             # Vector(4, 6)
```



Outline

1. Dunder (Magic) Methods
2. Overriding `__str__`
3. Overriding `__eq__`
4. Overriding `__lt__`
5. `__repr__`
6. Operator Overloading: `__add__`
- 7. Summary**



7. Summary

Override when...	Method
You want readable <code>print()</code> output	<code>__str__</code>
You want meaningful equality (<code>==</code>)	<code>__eq__</code>
You want <code>sort()</code> to work on your objects	<code>__lt__</code>
You want REPL to show useful info	<code>__repr__</code>
You want <code>+</code> , <code>-</code> , <code>*</code> to work	<code>__add__</code> , <code>__sub__</code> , <code>__mul__</code>

Thanks for Watching - Any questions?

